MODULE 15 · DOMAIN 5 - TERRAFORM MODULES

Module Versioning and the Registry

The version argument, publishing requirements, and why module pinning matters more than provider pinning.

DOMAIN 5 of 8 COURSE WEIGHT 10% MODULE 15 of 27

admin@sthithapragna.com

Providers are **constrained and then locked**. Modules are **constrained only**. The constraint is all you get.

The version argument

- **version** goes in the module block, alongside source. It uses the same operators as providers.
- **It works only with registry sources**, public or private. Nothing else accepts it.
- Put version on a Git or local source and Terraform **errors**. Those pin by ref, or not at all.
- Recall the operators from module three: equals, not equals, the comparisons, and the pessimistic form.
- Constraints from every module are combined, and Terraform selects **the newest release satisfying all of them**.

> ON THE EXAM
>
> Registry sources support version. Git sources support ref. Local paths support neither, because there is nothing to select from.

Which pin does which source take?

SOURCE	HOW YOU PIN IT
Public registry	version argument. Full operator set
Private registry	version argument. Identical behaviour
Generic Git	ref argument, naming a tag, branch or commit
GitHub or Bitbucket shorthand	ref argument, same as generic Git
HTTP, S3 or GCS archive	The URL itself. Point it at a versioned artefact
Local path	Nothing. Whatever is on disk is what runs

ON THE EXAM

Only the first two accept version. Everything else pins by naming one specific thing, or does not pin at all.

The consequence of no lock file

- **Terraform never records which module version it selected.** There is no module entry in the lock file.
- So on **every init**, it re-resolves and takes the newest release that satisfies your constraint.
- Two machines initialising on different days can therefore run **different module code**.
- HashiCorp's own remedy: **use an exact version constraint** if you need the same version every time.
- **A pessimistic constraint is not a pin.** It still allows the rightmost component to move.

> ON THE EXAM
>
> This is module three's finding, restated as advice. Providers are reproducible by default; modules are reproducible only if you pin them.

How strict should a module pin be?

EXACT VERSION

Reproducible, and manual

- Every environment runs the same code
- Upgrades are a deliberate commit
- Diffs show exactly what changed
- You must remember to move forward

PESSIMISTIC CONSTRAINT

Convenient, and drifting

- Patches arrive automatically
- No commit needed to take a fix
- Two environments may differ
- A plan can change with no code change

THE TIEBREAKER

The trade is different from providers, because there is no lock file underneath to make the loose option safe.

init, and moving a module version

- **terraform init installs modules** into the .terraform directory. One of its three jobs, from module five.
- **Changing a source or a version requires init again.** Plan will stop and tell you so.
- **terraform init -upgrade** re-resolves modules as well as providers against current constraints.
- **terraform get** installs modules only, without touching the backend or providers.
- Plain init **does not** reinstall a module that is already present and still satisfies the constraint.

> ON THE EXAM
>
> -upgrade means both things at once: newest matching providers, and newest matching modules. It is not provider-only.

Publishing to the public registry

- **GitHub, and the repository must be public.** This applies to the public registry only.
- **Named terraform, then the provider, then the name.** Three parts, separated by hyphens.
- **A repository description**, which becomes the module's one-sentence summary.
- **The standard module structure**, so the registry can inspect it and generate documentation.
- **Release tags in x.y.z form**, optionally prefixed with a v. At least one is needed to publish.

> ON THE EXAM
>
> Five requirements, and the naming one catches people. The name segment may contain further hyphens; the first two parts may not.

Releasing a new version

- **Push a properly formed Git tag.** That is the entire release process for a public module.
- The registry notices the tag and publishes that version. No separate upload step exists.
- **Tags that do not parse as a semantic version are ignored**, silently.
- So a release named release-2024-q3 will never appear, and nothing will tell you why.
- Versions are immutable once published. Fixing a mistake means **publishing a new version**.

> ON THE EXAM
>
> Publishing initially is a one-off setup. After that, a version is a tag, which is why tag discipline becomes release discipline.

What the registry gives you back

- **Generated documentation** from your variables, outputs and README. Descriptions become the docs.
- **A browsable version history**, and a usage example you can copy.
- **Inputs, outputs, dependencies and provider requirements**, all parsed from the source.
- **Submodules and examples are parsed too**, which is why the standard structure is a requirement.
- All of it is available through the **UI and the API**.

> ON THE EXAM
>
> The structure requirement is not bureaucracy. The registry reads those directories to produce everything on this slide.

The private registry

- HCP Terraform and Terraform Enterprise both provide a **private module registry** for an organisation.
- **The public GitHub requirement does not apply.** That one is specific to the public registry.
- Publishing from version control with tag-based releases still requires **x.y.z release tags**.
- The address gains a hostname, so a private module reference has **four parts instead of three**.
- Modules can be marked **no-code ready**, for provisioning without writing configuration. Module twenty-three.

> ON THE EXAM
>
> Private registry modules use the version argument exactly as public ones do. Only the address and the hosting differ.

The advantage people miss

- **The private registry downloads and stores the module source code itself.**
- It serves downloads and controls access using **HCP Terraform API tokens**.
- **So consumers do not need access to the module's source repository**, even running from the CLI.
- With a Git source, every consumer needs credentials for that repository. Every pipeline too.
- That difference is often the real reason to move from Git sources to a private registry.

> ON THE EXAM
>
> Versioning is the obvious benefit. Decoupling module consumption from source repository permissions is the quieter one.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Terraform rejected the version argument	The source is not a registry. Use ref instead
The same config gave different results	An unpinned module, re-resolved on init
Every environment must run identical code	An exact version constraint. There is no lock file
The module never appeared in the registry	Naming, or a tag that is not a semantic version
Consumers need repo credentials to init	A Git source. A private registry removes that
A fix was published but nobody picked it up	An exact pin, working as intended

ON THE EXAM

Half these questions are decided by remembering there is no module lock file. The other half by the five publishing requirements.

> A regulated team must be able to demonstrate that the code deployed to production is byte-for-byte the code that was reviewed. They pin every provider with a pessimistic constraint and commit the dependency lock file.
>
> They use registry modules with the same pessimistic constraints, reasoning that the approach worked for providers.

What is wrong with this, and what should change?

A. Nothing. The lock file records module selections alongside providers

B. Modules are not locked. Use exact version constraints for the modules

C. Switch the modules to Git sources, which are locked by commit hash

D. Run terraform init -upgrade before every apply so all environments match

◆ PAUSE AND DECIDE

What is wrong with this, and what should change?

A The lock file covers modules

It does not, and never has. HashiCorp states the lock file tracks only provider dependencies.

B Pin modules exactly

Correct. With no lock file underneath, an exact constraint is the only mechanism that guarantees the same module version every time.

C Use Git sources

A ref can name a commit, so this can work. It also loses the version argument and the registry, to solve something an exact pin solves already.

D -upgrade everywhere

The opposite of what they need. It re-resolves to the newest matching release every time, maximising the drift they are trying to eliminate.

THE SIGNAL

The reasoning that failed them is the reasonable one: what worked for providers should work for modules. The lock file is the difference.

> An engineer prepares a module for the public registry. The repository is public on GitHub, follows the standard module structure, and has a description. She tags the release as release-2026-03 and clicks publish.
>
> The registry accepts the repository but shows no available versions.

What is the problem?

A. The repository must be named with a terraform prefix, provider and name

B. Release tags must be a semantic version, optionally prefixed with v. Other tags are ignored

C. A module needs at least three released versions before any are shown

D. The publish step requires a signed tag for verification

◆ PAUSE AND DECIDE

What is the problem?

A Naming

A real requirement, and ruled out here: the registry accepted the repository, which it would not have done had the name been wrong.

B The tag is not a version

Correct. The registry identifies versions from tags in x.y.z form, optionally prefixed with v. Anything else is ignored silently.

C Three versions needed

Invents a rule. One release tag is enough, and at least one is required to publish initially.

D Signed tags

Provider releases involve signing. Module publishing has no such requirement.

THE SIGNAL

The registry accepted the repository and found no versions. That split tells you the naming passed and the tags did not.

> A platform team distributes modules from a private Git repository. Every consuming team needs a deploy key, and every pipeline needs credentials for that repository. Onboarding a new team takes two days, mostly spent on access requests.
>
> They already use HCP Terraform for their runs.

What would most reduce the onboarding burden?

A. Make the module repository readable by everyone in the organisation

B. Publish the modules to the HCP Terraform private registry

C. Vendor the modules into each consuming repository with a sync job

D. Move to local path sources so no credentials are needed

◆ PAUSE AND DECIDE

What would most reduce the onboarding burden?

A. Open the repository

Reduces the requests and grants write-adjacent visibility to everyone, and still requires each pipeline to hold VCS credentials.

B. Use the private registry

Correct. The registry stores the source and serves downloads via API tokens, so consumers need no access to the module repository at all.

C. Vendor with a sync job

Module fourteen's divergence problem, deliberately reintroduced. Every consumer holds a copy that can drift.

D. Local path sources

Local paths cannot cross repository boundaries in any reliable way, and they support no version argument at all.

THE SIGNAL

The registry decouples module consumption from source repository permissions. That is often the real reason teams adopt it.

> A configuration sources a module from a generic Git repository with a ref argument pinning it to a tag. An engineer adds a version argument set to a pessimistic constraint, wanting the same behaviour registry modules have.
>
> terraform init fails.

Why, and what achieves the intent?

A. The version argument works only with registry sources. Change the ref, or move to a registry

B. The version and ref arguments conflict. Remove ref and keep version

C. Git sources need a version_constraint argument rather than version

D. The constraint syntax differs for modules; only exact versions are allowed

◆ PAUSE AND DECIDE

Why, and what achieves the intent?

A Registry sources only

Correct. A registry publishes a list of versions to choose among; a Git URL does not. Achieving range-based selection means publishing to a registry.

B They conflict

Implies version would work alone on a Git source. It would not. The source type is what makes it invalid, not the combination.

C version_constraint

Invents an argument. There is no such thing on a module block.

D Only exact versions

Backwards. Registry modules accept the full operator set; the problem is that this source is not a registry.

THE SIGNAL

A constraint needs a set of candidates to choose from. A registry publishes one; a Git ref names a single commit and nothing else.

Module 15 recap

- **version works with registry sources only.** Git pins by ref; local paths pin not at all.
- **Modules are never recorded in the lock file**, so a constraint is all the pinning you get.
- **Use an exact version** where reproducibility matters. That is HashiCorp's own advice.
- **Five publishing requirements:** public GitHub, three-part name, description, standard structure, x.y.z tags.
- **A release is a tag.** Tags that are not semantic versions are ignored silently.
- **The private registry stores the source**, so consumers need no repository access.

> ON THE EXAM
>
> terraform init -upgrade re-resolves modules as well as providers. It is not a provider-only flag.